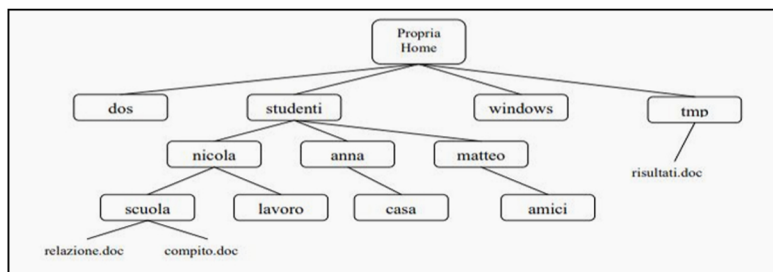


Emy Mangogna - W5D1 - Pratica

Ex. 1 - PT 1

➤ Creazione di cartelle, sottocartelle e file testuali all'interno della directory **Home** attraverso comandi dal terminale come in figura.



Utilizzo del comando **mkdir** (make directory) per creare le cartelle "dos", "studenti" "windows" e "tmp";
il comando **ls -l** mostra l'elenco di tutte le cartelle presenti all'interno della directory **Home** e le loro caratteristiche.

```
(kali㉿kali)-[~]
└─$ mkdir dos studenti windows tmp

(kali㉿kali)-[~]
└─$ ls -l
total 40
drwxrwxr-x 2 kali kali 4096 Jul 26 12:42 Desktop
drwxr-xr-x 2 kali kali 4096 Jun 24 12:45 Documents
drwxrwxr-x 2 kali kali 4096 Jul 26 12:48 dos
drwxr-xr-x 2 kali kali 4096 Jul 25 14:23 Downloads
-rw-r--r-- 1 kali kali 686 Jul 14 16:49 nohup.out
drwxr-xr-x 2 kali kali 4096 Jun 24 12:45 Public
drwxrwxr-x 2 kali kali 4096 Jul 26 12:48 studenti
drwxr-xr-x 2 kali kali 4096 Jun 24 12:45 Templates
drwxrwxr-x 2 kali kali 4096 Jul 26 12:48 tmp
drwxrwxr-x 2 kali kali 4096 Jul 26 12:48 windows
```

Il comando **mkdir -p** permette di creare tutte le sottocartelle di "studenti",
-p (parent) permette di creare cartelle intermedie nel percorso specificato.
Il comando **tree** mostra tutta la struttura della directory "**studenti**" con cartelle e sottocartelle appena create.

```
(kali㉿kali)-[~]
└─$ mkdir -p studenti/nicola/lavoro studenti/nicola/scuola studenti/anna/casa studenti/matteo/amici

(kali㉿kali)-[~]
└─$ tree studenti
studenti
├── anna
│   └── casa
├── matteo
│   └── amici
└── nicola
    ├── lavoro
    └── scuola

8 directories, 0 files
```

Il comando **touch** permette di creare un nuovo file vuoto (in questo caso, files di testo .doc).

```
(kali@kali)-[~]
$ touch studenti/nicola/scuola/relazione.doc studenti/nicola/scuola/compito.doc tmp/risultati.doc

(kali@kali)-[~]
$ tree ~
/home/kali
├── Desktop
├── Documents
├── dos
├── Downloads
│   ├── 2717-space-stars-starry-sky-dark-4k.jpg
│   ├── code_1.102.2-1753187809_amd64.deb
│   ├── esercizio_w5d5
│   ├── Esercizio_w5D5
│   └── esercizio_w5d5.c
├── nohup.out
├── Public
├── studenti
│   ├── anna
│   │   ├── casa
│   │   ├── matteo
│   │   └── amici
│   ├── nicola
│   │   ├── lavoro
│   │   └── scuola
│   │       ├── compito.doc
│   │       └── relazione.doc
├── Templates
├── tmp
│   └── risultati.doc
└── windows

17 directories, 9 files
```

Utilizzaando dinuovo il comando **tree**, vengono mostrate tutte le cartelle, sottocartelle e file di vario tipo (.doc, .jpg, .deb, .c) presenti all'interno della directory **Home**.

Ex.1 - PT 2

➤ Partendo dalla **directory lavoro** (~/**studenti/nicola/lavoro**) spostarsi nella cartella "casa" sotto "anna" utilizzando **percorso relativo** e **percorso assoluto**.

```
(kali@kali)-[~]
$ pwd
/home/kali

(kali@kali)-[~]
$ cd ~/studenti/nicola/lavoro

(kali@kali)-[~/studenti/nicola/lavoro]
$ cd ../../anna/casa

(kali@kali)-[~/studenti/nicola/lavoro]
$ cd ~/studenti/anna/casa

(kali@kali)-[~/studenti/anna/casa]
$
```

Il comando **pwd** (print working directory) mostra la posizione attuale (**home**), con il comando **cd ~/studenti/nicola/lavoro** ci si sposta nella cartella richiesta dall'esercizio; **cd** (change directory), **~** (shortcut per la **home**).

Utilizzando **cd ../../anna/casa** si raggiungere la cartella "casa", effettuando uno spostamento con **percorso relativo**. **".."** indica la directory superiore (genitore) in questo caso utilizzato due volte perche si deve salire di due livelli.

Il comando **cd ~/studenti/anna/casa** permette di spostatarsi direttamente da "home" a "casa" con **persorco assoluto**.

- > **Copiare** il file compito.doc dalla cartella "scuola" alla cartella "casa".
- > **Spostare** il file relazione.doc dalla cartella "scuola" alla cartella "casa".
- > **Cancellare** la cartella "tmp".

```
(kali㉿kali)-[~/studenti/anna/casa]
$ cp ~/studenti/nicola/scuola/compito.doc .
(kali㉿kali)-[~/studenti/anna/casa]
$ ls
compito.doc
(kali㉿kali)-[~/studenti/anna/casa]
$ mv ~/studenti/nicola/scuola/relazione.doc .
(kali㉿kali)-[~/studenti/anna/casa]
$ ls
compito.doc  relazione.doc
(kali㉿kali)-[~/studenti/anna/casa]
$ rm -r ~/tmp
(kali㉿kali)-[~/studenti/anna/casa]
$ ls ~
Desktop  Documents  dos  Downloads  nohup.out  Public  studenti  Templates  windows
```

Il comando **cp ~/studenti/nicola/scuola/compito.doc .** permette di copiare il file .doc dalla cartella di provenienza e spostarlo direttamente nella cartella "casa";

cp (copy) e **"."** (cartella corrente).

Con **ls** verifichiamo che il file si trovi nella giusta directory.

Il comando **mv ~/studenti/nicola/scuola/relazione.doc .** permette di spostare il file .doc dalla cartella di provenienza direttamente nella cartella "casa";

mv (move) e **"."** (cartella corrente).

Anche in questo caso si verifica che il file si trovi nella giusta directory con il comando **ls**.

Il comando **rm -r ~/tmp** permette di rimuovere la cartella "tmp" dalla **home**.

rm -r(remove recursive) elimina file e directory;

l'opzione **-r** (ricorsiva) è necessaria per cancellare cartelle che contengono file, come tmp.

In fine con **ls** visualizziamo le cartelle contenute in **home** e "tmp" non è più presente.

> **Creare** il file **pippo.txt** nella cartella “lavoro”.

> **Modificare** i permessi di **pippo.txt**:

- proprietario: **lettura e scrittura**
- altri utenti: **sola lettura**

```
(kali㉿kali)-[~/studenti/anna/casa]
$ cd ~/studenti/nicola/lavoro

(kali㉿kali)-[~/studenti/nicola/lavoro]
$ touch pippo.txt

(kali㉿kali)-[~/studenti/nicola/lavoro]
$ ls
pippo.txt

(kali㉿kali)-[~/studenti/nicola/lavoro]
$ chmod 644 pippo.txt

(kali㉿kali)-[~/studenti/nicola/lavoro]
$ ls -l pippo.txt
-rw-r--r-- 1 kali kali 0 Jul 27 18:33 pippo.txt
```

Il comando **cd ~/studenti/nicola/lavoro** ci porta nella cartella lavoro, qui usiamo il comando **touch pippo.txt** per creare un file vuoto chiamato “pippo.txt” nella cartella corrente.

Con **ls** visualizziamo i file presenti, per verificare che **pippo.txt** sia stato creato.

Il comando **chmod 644 pippo.txt** rende il file “pippo.txt” leggibile e scrivibile solo dal proprietario (emy), mentre tutti gli altri utenti devono poterlo solo leggere;

chmod (change mode) modifica i permessi di un file.

644 = schema permessi:

6 (rw-) per il proprietario - lettura e scrittura.

4 (r--) per il gruppo - sola lettura.

4 (r--) per gli altri - sola lettura.

ls -l pippo.txt mostra che i permessi sono stati cambiati correttamente

- > **Nascondere** il contenuto della cartella “anna”.
- > Spostarsi nella cartella lavoro e **visualizzare il contenuto** del file pippo.txt.

```
(kali@kali)-[~/studenti/nicola/lavoro]
$ chmod 000 ~/studenti/anna

(kali@kali)-[~/studenti/nicola/lavoro]
$ ls ~/studenti/anna
ls: cannot open directory '/home/kali/studenti/anna': Permission denied

(kali@kali)-[~/studenti/nicola/lavoro]
$ cd ~/studenti/nicola/lavoro

(kali@kali)-[~/studenti/nicola/lavoro]
$ cat pippo.txt

(kali@kali)-[~/studenti/nicola/lavoro]
$
```

Il comando **chmod 000 ~/studenti/anna** rimuove ogni permesso di lettura, scrittura ed esecuzione dalla cartella “anna”.

chmod(change mode) modifica i permessi di un file o directory, **000** (toglie tutti i permessi r w x).

Andando a verificare con **ls**, appare il messaggio “**Permission denied**”, confermando che l’accesso è bloccato.

Con il comando **cd ~/studenti/nicola/lavoro** si cambia la directory per tornare nella cartella “lavoro”.

cat pippo.txt mostra il contenuto del file “pippo.txt” direttamente nel terminale. Siccome il file è vuoto, non appare nulla.

- > **Rimuovere** la cartella amici.
- > **Eliminare** tutte le cartelle create: dos, studenti, windows, tmp.

```
(kali@kali)-[~/studenti/nicola/lavoro]
$ ls ~/studenti/matteo

(kali@kali)-[~/studenti/nicola/lavoro]
$ tree ~/studenti
/home/kali/studenti
├── anna [error opening dir]
├── matteo
├── nicola
│   ├── lavoro
│   │   ├── pippo.txt
│   │   └── scuola
│   └── compito.doc
6 directories, 2 files
```

Il comando **rm -r ~/studenti/matteo/amici** permette eliminare completamente la cartella amici e il suo contenuto.

Con **ls ~/studenti/matteo** si verifica che la cartella “amici” è stata effettivamente cancellata.

Utilizzando il comando **tree ~/studenti** viene mostrata la struttura della directory dopo le modifiche apportate.

Il messaggio **[error opening dir]** appare perchè **tree** ha provato ad aprire la cartella “anna”, ma non ha i permessi per farlo.

```
(kali@kali)-[~/studenti/nicola/lavoro]
$ rm -r ~/dos ~/studenti ~/windows
rm: cannot remove '/home/kali/studenti/anna': Permission denied

(kali@kali)-[~/studenti/nicola/lavoro]
$ ls ~
Desktop Documents Downloads nohup.out Public |studenti| Templates

(kali@kali)-[~/studenti/nicola/lavoro]
$ chmod 755 ~/studenti/anna

(kali@kali)-[~/studenti/nicola/lavoro]
$ rm -r ~/studenti

(kali@kali)-[~/studenti/nicola/lavoro]
$ ls ~
Desktop Documents Downloads nohup.out Public Templates
```

Usando il comando **rm -r ~/dos ~/studenti ~/windows** si può procedere con l'eliminazione di tutte le cartelle presenti su **home**, ma il comando fallisce perché la cartella "anna" dentro studenti ha permessi **000** (nessun permesso) e quindi non può rimuovere la cartella "studenti".

Per eliminare tutte le cartelle presenti su **home** è necessario ripristinare i permessi di lettura/esecuzione con **chmod 755 ~/studenti/anna;**
755 (7 = r+w+x 5 = r+x 5 = r+x).

Adesso il comando **rm -r ~/dos ~/studenti ~/windows** funziona senza errori perché anna è accessibile.

Con **ls ~** possiamo verificare che le cartelle sono state eliminate.

> Ex.2 - Facoltativo:

- Provare i comandi **w**, **who**, **whoami**.
- Leggere il manuale dei comandi **jobs**, **ps**, **kill**.
- Lanciare il comando **vi pippo**.
- Aprire un nuovo terminale e visualizza tutti i propri processi.
- Terminare (killare) il processo "vi" per sbloccare il primo terminale.
- Lanciare il comando Firefox in foreground*.
- Portarlo in background.
- Terminare il processo Firefox.

```
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)-[~]
$ w
09:39:51 up 3 min,  1 user,  load average: 0.67, 0.80, 0.37
USER      TTY      FROM          LOGIN@   IDLE   JCPU   PCPU   WHAT
kali             -                09:37    IDLE   0.00s   0.01s  lightdm --session-child 13 24

(kali@kali)-[~]
$ who
kali      seat0      2025-07-28 09:37 (:0)

(kali@kali)-[~]
$ whoami
kali
```

Il comando **w** mostra chi è collegato al sistema e le attività di ogni utente (terminale usato, da quanto tempo è loggato, inattività, processi in esecuzione)

il comando **who** mostra gli utenti connessi al sistema, con informazioni di base (nome, terminale, data/ora login)

whoami restituisce il nome dell'utente con cui si é attualmente loggati.

```
JOB CONTROL
Job control refers to the ability to selectively stop (suspend) the execution of processes and continue (resume) their execution at a later point. A user typically employs this facility via an interactive interface supplied jointly by the operating system kernel's terminal driver and bash.
```

```
PS(1)                                User Commands                                PS(1)
NAME
ps - report a snapshot of the current processes.
SYNOPSIS
ps [options]
DESCRIPTION
ps displays information about a selection of the active processes. If you want a repetitive update of the selection and the displayed information, use top instead.
```

```
KILL(1)                                User Commands                                KILL
NAME
kill - send a signal to a process
SYNOPSIS
kill [options] <pid> [ ... ]
DESCRIPTION
The default signal for kill is TERM. Use -l or -L to list available signals. Particularly useful signals include HUP, INT, KILL, STOP, CONT, and 0. Alternate signals may be specified in three ways: -SIGKILL or -KILL. Negative PID values may be used to choose whole process groups; see the PGID column in ps command output. A PID of -1 is special; it indicates all processes except the kill process itself and init.
```

I comandi **man jobs**, **man ps** e **man kill**, mostrano rispettivamente le sezioni del manuale dei comandi, dove viene mostrata la sintassi e le opzioni disponibili

* La traccia dava: 6. Lanciare il comando firefox in background e 7. portarlo in background. Ho dato per scontato fosse un errore.

Il comando **vi pippo** avvia l'editor vi per creare/modificare il file pippo. Il programma occupa il terminale come processo in esecuzione, bloccandolo finché non viene chiuso. Apriamo un secondo terminale per gestire (terminare) il processo e sbloccare il primo.

ps mostra i processi attivi nel terminale corrente.

ps aux | grep vi mostra l'elenco dei processi e il loro PID (Process ID), che serve per identificarli in modo univoco. (**PID 3797**)

grep serve a filtrare solo quelli che interessano (in questo caso **vi**)

Con **kill 3797** si termina il processo "vi pippo".

Dando nuovamente il comando **ps aux | grep vi** si può verificare che il processo vi pippo non c'è più, questo significa che si è sbloccato il primo terminale terminando l'editor vi che lo occupava.

```
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)-[~]
$ ps
  PID TTY          TIME CMD
 3922 pts/1        00:00:00 zsh
 3995 pts/1        00:00:00 ps

(kali@kali)-[~]
$ ps aux | grep vi
root      612   0.0   0.1 291740   3500 ?        Ssl   10:28   0:00 /usr/sbin/VBoxService
kali     1013   0.0   0.1 155452   3600 ?        Ssl   10:33   0:00 /usr/bin/gpg-agent --supervised
kali     1045   0.0   0.3 165232   6096 ?        Ssl   10:33   0:00 /usr/libexec/dconf-service
kali     1069   0.0   1.2 345976  25260 ?        Sl    10:33   0:00 /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_64-linux-gnu/xfce4/panel/plugins/libsystray.so 14 27262989 systray Status Tray Plugin Provides status notifier items (application indicators) and legacy systray items
kali     1073   0.0   1.9 286516  38608 ?        Sl    10:33   0:00 /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_64-linux-gnu/xfce4/panel/plugins/libxfce4powermanager.so 18 27262993 power-manager-plugin Power Manager Plugin Display the battery levels of your devices and control the brightness of your display
kali     3797   0.1   0.5 16212  10760 pts/0    Sl+   10:38   0:00 vi pippo
kali     4175   0.0   0.1   6528   2256 pts/1    S+    10:38   0:00 grep --color=auto vi

(kali@kali)-[~]
$ kill 3797

(kali@kali)-[~]
$ ps aux | grep vi
root      612   0.0   0.1 291740   3500 ?        Ssl   10:28   0:00 /usr/sbin/VBoxService
kali     1013   0.0   0.1 155452   3600 ?        Ssl   10:33   0:00 /usr/bin/gpg-agent --supervised
kali     1045   0.0   0.3 165232   6096 ?        Ssl   10:33   0:00 /usr/libexec/dconf-service
kali     1069   0.0   1.2 345976  25260 ?        Sl    10:33   0:00 /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_64-linux-gnu/xfce4/panel/plugins/libsystray.so 14 27262989 systray Status Tray Plugin Provides status notifier items (application indicators) and legacy systray items
kali     1073   0.0   1.9 286516  38608 ?        Sl    10:33   0:00 /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_64-linux-gnu/xfce4/panel/plugins/libxfce4powermanager.so 18 27262993 power-manager-plugin Power Manager Plugin Display the battery levels of your devices and control the brightness of your display
kali     5079   0.0   0.1   6528   2204 pts/1    S+    10:40   0:00 grep --color=auto vi
```



```

zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)-[~]
$ firefox
^Z
zsh: suspended  firefox

(kali@kali)-[~]
$ bg
[1] + continued  firefox

```

Col comando **firefox** il browser viene avviato in foreground, il programma si apre e occupa il terminale.

Ctrl + Z sospende il processo e libera il terminale senza chiudere Firefox

Il comando **bg** riattiva Firefox in background. Ora Firefox continua a funzionare, ma il terminale è libero per altri comandi.

```

(kali@kali)-[~]
$ ps aux | grep firefox
kali 39214 1.7 15.9 2895732 321880 pts/0 Sl 11:50 0:09 firefox-esr
kali 39260 0.0 2.1 214800 43960 pts/0 Sl 11:50 0:00 /usr/lib/firefox-esr/firefox-esr -contentproc
-parentBuildID 20250519114620 -prefsLen 28493 -prefMapSize 249416 -appDir /usr/lib/firefox-esr/browser {fcd95805-
953e-476d-a259-cf5c23849d20} 39214 true socket
kali 39281 0.1 5.5 2423680 112380 pts/0 Sl 11:50 0:00 /usr/lib/firefox-esr/firefox-esr -contentproc
-childID 1 -isForBrowser -prefsLen 28634 -prefMapSize 249416 -jsInitLen 234912 -parentBuildID 20250519114620 -gre
omni /usr/lib/firefox-esr/omni.ja -appomni /usr/lib/firefox-esr/browser/omni.ja -appDir /usr/lib/firefox-esr/brow
ser {b6c5f560-5808-4c69-a714-a7aafca59caf} 39214 true tab
kali 39346 0.1 4.8 2432444 97656 pts/0 Sl 11:50 0:00 /usr/lib/firefox-esr/firefox-esr -contentproc
-childID 2 -isForBrowser -prefsLen 34055 -prefMapSize 249416 -jsInitLen 234912 -parentBuildID 20250519114620 -gre
omni /usr/lib/firefox-esr/omni.ja -appomni /usr/lib/firefox-esr/browser/omni.ja -appDir /usr/lib/firefox-esr/brow
ser {83834001-5db3-47fd-b0fa-bcc538180da5} 39214 true tab
kali 39370 0.1 6.4 2465852 131072 pts/0 Sl 11:50 0:01 /usr/lib/firefox-esr/firefox-esr -contentproc
-childID 3 -isForBrowser -prefsLen 31455 -prefMapSize 249416 -jsInitLen 234912 -parentBuildID 20250519114620 -gre
omni /usr/lib/firefox-esr/omni.ja -appomni /usr/lib/firefox-esr/browser/omni.ja -appDir /usr/lib/firefox-esr/brow
ser {7a64fcb-d407-499d-b0b9-83367143e2ef} 39214 true tab
kali 39440 0.0 2.0 212000 40716 pts/0 Sl 11:50 0:00 /usr/lib/firefox-esr/firefox-esr -contentproc
-parentBuildID 20250519114620 -sandboxingKind 0 -prefsLen 37677 -prefMapSize 249416 -appDir /usr/lib/firefox-esr/
browser {378f8a36-7b31-4ca1-a3c7-30f8ab560375} 39214 true utility
kali 39444 0.0 3.6 2399068 74568 pts/0 Sl 11:50 0:00 /usr/lib/firefox-esr/firefox-esr -contentproc
-childID 4 -isForBrowser -prefsLen 31455 -prefMapSize 249416 -jsInitLen 234912 -parentBuildID 20250519114620 -gre
omni /usr/lib/firefox-esr/omni.ja -appomni /usr/lib/firefox-esr/browser/omni.ja -appDir /usr/lib/firefox-esr/brow
ser {07a1bebe-b092-49a4-a2e9-bf4fb896559f} 39214 true tab
kali 39446 0.0 3.6 2399068 74612 pts/0 Sl 11:50 0:00 /usr/lib/firefox-esr/firefox-esr -contentproc
-childID 5 -isForBrowser -prefsLen 31455 -prefMapSize 249416 -jsInitLen 234912 -parentBuildID 20250519114620 -gre
omni /usr/lib/firefox-esr/omni.ja -appomni /usr/lib/firefox-esr/browser/omni.ja -appDir /usr/lib/firefox-esr/brow
ser {659b5fc4-d0dc-4a5b-ba06-acd598acc952} 39214 true tab
kali 39470 0.0 3.6 2399068 74384 pts/0 Sl 11:50 0:00 /usr/lib/firefox-esr/firefox-esr -contentproc
-childID 6 -isForBrowser -prefsLen 31455 -prefMapSize 249416 -jsInitLen 234912 -parentBuildID 20250519114620 -gre
omni /usr/lib/firefox-esr/omni.ja -appomni /usr/lib/firefox-esr/browser/omni.ja -appDir /usr/lib/firefox-esr/brow
ser {1f93042f-521e-4fdd-925b-f7ed52972499} 39214 true tab
kali 43818 0.0 0.1 6528 2348 pts/0 S+ 11:59 0:00 grep --color=auto firefox

(kali@kali)-[~]
$ kill 39214

Exiting due to channel error.
Exiting due to channel error.
Exiting due to channel error.
Exiting due to channel error.
Exiting due to channel error.
Exiting due to channel error.
[1] + terminated firefox

(kali@kali)-[~]
$

```

Per terminare Firefox si deve individuare l'ID del processo (PID); con il comando **ps aux | grep firefox** si visualizzano tutti i processi in esecuzione e si possono filtrare quelli che contengono "firefox" nel nome.

Ottenuto il PID (39214) è possibile terminare Firefox col comando **kill 39214**.

Il programma e tutti i suoi sottoprocessi sono stati chiusi, come mostrano gli errori **"Exiting due to channel error"** (messaggio che viene mostrato quando i processi figli perdono il genitore).